Part3. 单元测试JUnit与质量度量

从入门到精通的技术实践指南

何楚

CONTENTS

01 单元测试导论

理解单元测试的本质、原则与价值。

02 JUnit 核心实战

掌握JUnit 的注解、断言与运行器。

03 JUnit 运行原理

深入解析JUnit 的架构与执行流程。

04 代码覆盖率

学习如何度量和分析代码覆盖率。

05 变异测试

探索更高级的测试质量评估方法。

06 案例与代码示例

通过实战案例巩固所学知识。

07 总结与展望

回顾核心要点并展望未来发展。

Let's dive into the world of unit testing.

什么是单元测试？

定义

单元测试（Unit Testing）是指对软件中的最小可测试单元进行检查和验证。

核心特征

最小单元

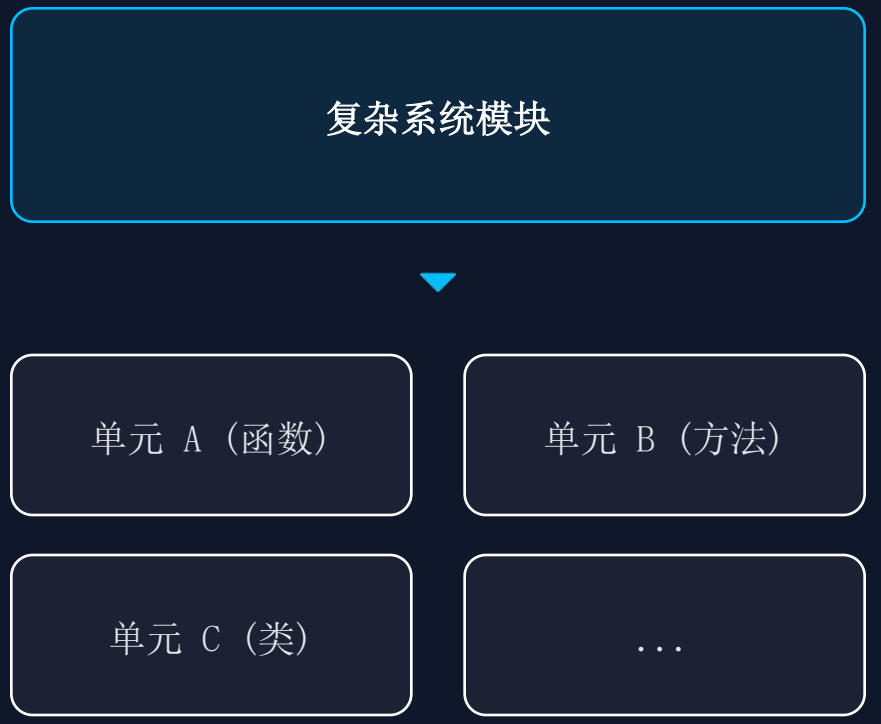
通常是一个函数、方法或类。

独立运行

测试用例相互隔离，不依赖外部环境。

自动化

测试单元拆解示意



为何要做单元测试？



早期发现缺陷

在开发过程中尽早发现并修复bug，大大降低后期修复成本。



提升代码质量

迫使开发者编写更具可读性、可维护性和低耦合的代码。



简化重构与维护

为代码修改提供安全网，确保重构不会引入新的问题。

Java 示例（被测试对象）

```java
// Calculator.java - 这是我们要测试的类
public class Calculator {  4个用法

    // 我们要测试的方法：加法
    public int add(int a, int b) {  1个用法
        return a + b;
    }

    // 另一个要测试的方法：除法
    public double divide(int a, int b) {  1个用法
        if (b == 0) {
            throw new IllegalArgumentException("除数不能为零");
        }
        return (double) a / b;
    }
}
```

测试用例

```java
import org.junit.Test;
import static org.junit.Assert.*;

// CalculatorTest.java - 这是测试类
public class CalculatorTest {

    @Test
    public void testAdd() {
        // 1. 准备阶段：创建对象，准备测试数据
        Calculator calculator = new Calculator();
        int a = 2;
        int b = 2;

        // 2. 执行阶段：调用被测试的方法
        int result = calculator.add(a, b);

        // 3. 断言阶段：验证结果是否符合预期
        assertEquals( expected: 4, result); // 期望是4，实际结果是result
    }
```

导入 JUnit 包

JUnit 注解，标记这是一个测试方法
JUnit 运行时会自动执行所有带此注解的方法
方法必须是 public void，不能有参数

核心断言方法
第一个参数 4：期望值
第二个参数 result：实际值
如果两者相等 → 测试通过
如果两者不等 → 测试失败

为什么要写单元测试？

```
// 后来你修改了加法逻辑...
public int add(int a, int b) {
    return a + b + 1; // 不小心改出了bug
}
// 运行测试后立即发现：
// testAdd() 失败！期望4，但得到的是5
```

单元测试的 FIRST 原则



F (Fast)

测试执行速度要快，以便频繁运行。



I (Independent)

测试用例之间相互独立，无依赖。



R (Repeatable)

测试结果应可重复，不受环境影响。



S (Self-Validating)

测试应能自动判断结果，无需人工干预。



T (Timely)

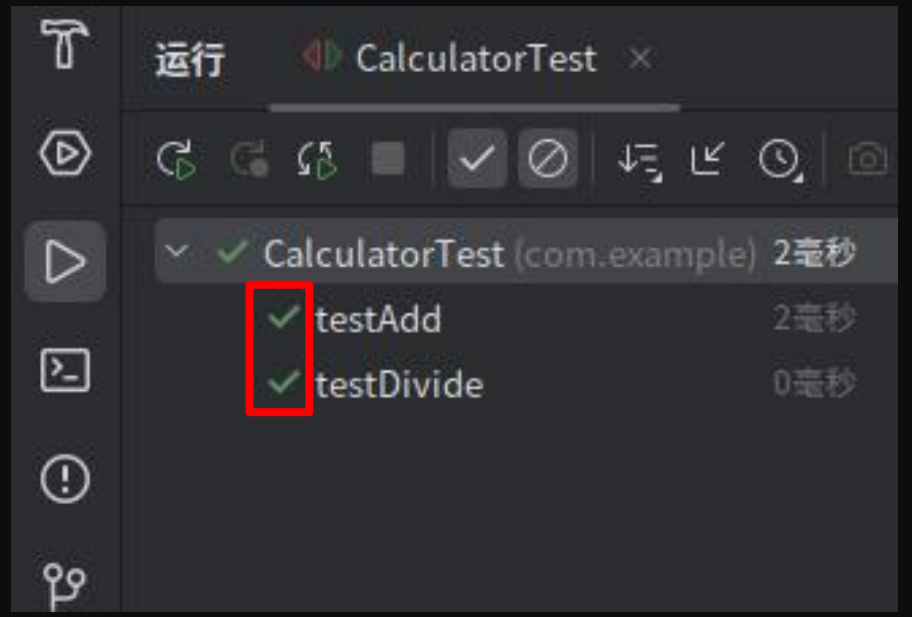
测试应在代码编写后及时编写。

JUnit 核心介绍

掌握单元测试利器

实际开发中的工作流程

- 编写业务代码（如 Calculator 类）
- 编写测试代码（如 CalculatorTest 类）
- 运行测试（IDE中右键 → Run Tests）
- 看到绿色对勾 ✓ = 所有测试通过
- 修改代码后，再次运行测试，确保还是绿色



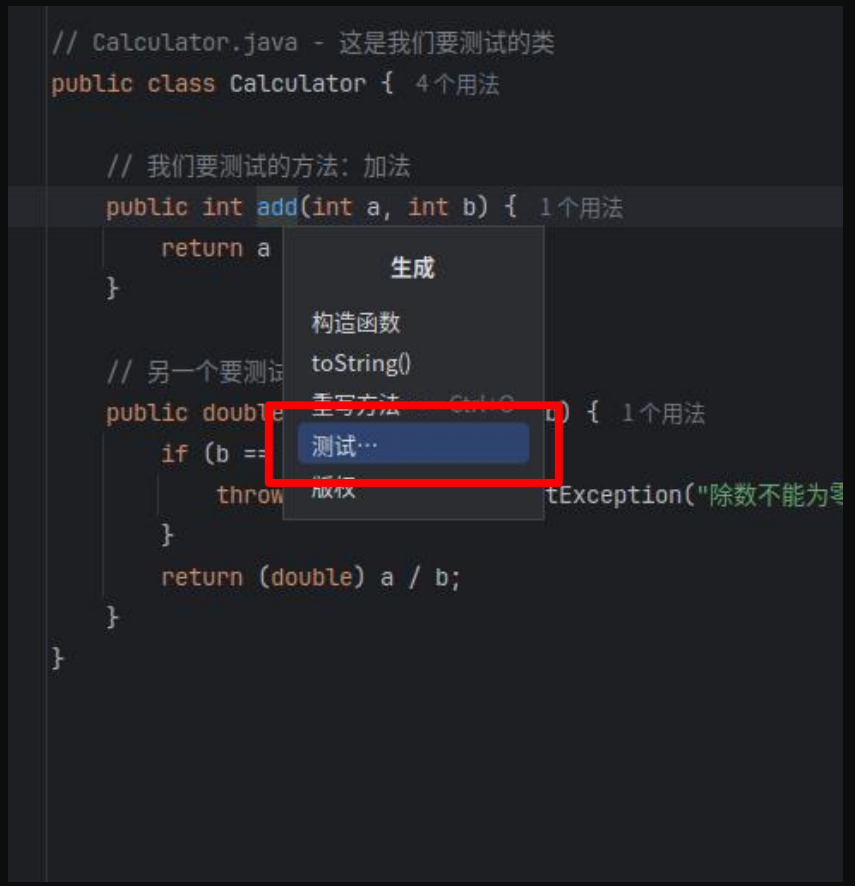
pom.xml 文件添加依赖

- <dependencies>
- <dependency>
- <groupId>junit</groupId>
- <artifactId>junit</artifactId>
- <version>4.13.1</version>
- <scope>test</scope>
- </dependency>
- </dependencies>

```
7    <dependencies>
8        <dependency>
9            <groupId>junit</groupId>
10           <artifactId>junit</artifactId>
11           <version>4.13.1</version>
12           <scope>test</scope>
13       </dependency>
14   </dependencies>
```

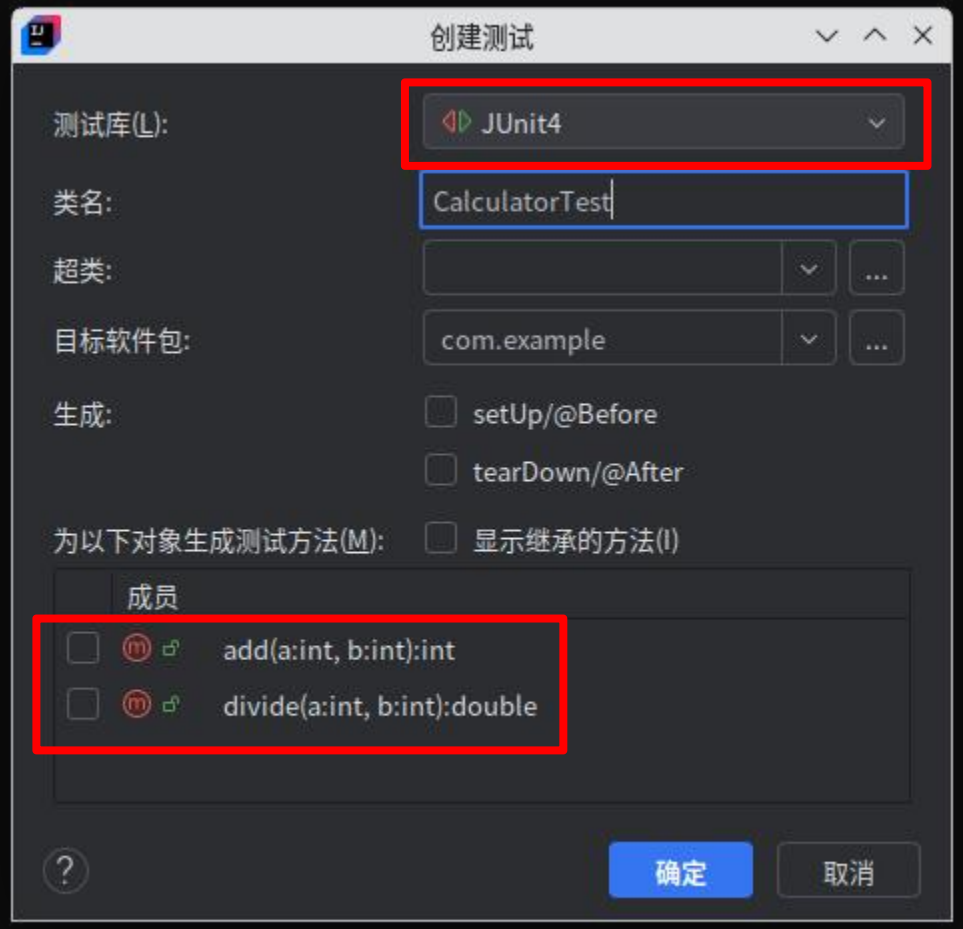
如何创建测试用例


// Calculator.java - 这是我们要测试的类
public class Calculator { 4个用法
// 我们要测试的方法：加法
public int add
return a
}
// 另一个要测试
public double
if (b == 0
throw
}
return (do
}
}
显示上下文操作 Alt+Enter
粘贴(P) Ctrl+V
复制/粘贴特殊
列选择模式(M) Alt+Shift+Insert
查找用法(U) Alt+Shift+7
转到
折叠
分析(Z)
重命名(R)... Shift+F6
生成... Alt+Insert
打开于
本地历史记录(H)
与剪贴板比较(B)
不能为零");

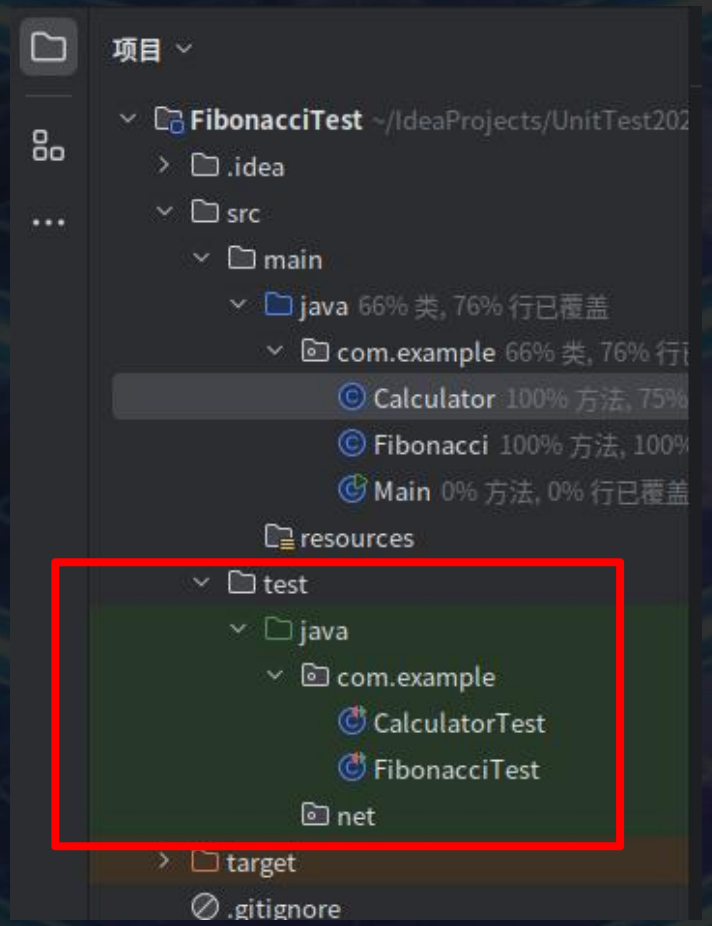

// Calculator.java - 这是我们要测试的类
public class Calculator { 4个用法
// 我们要测试的方法：加法
public int add(int a, int b) { 1个用法
return a
}
// 另一个要测试
public double
if (b ==
throw
}
return (double) a / b;
}
}
生成
构造函数
toString()
测试...
tException("除数不能为零

如何创建测试用例

选中要测试的函数



生成的测试用例在 src/test 目录



JUnit 核心注解

- **@Test**
  标记一个方法为测试方法。
- **@Before**
  在每个测试方法执行前都会运行。
- **@After**
  在每个测试方法执行后都会运行。
- **@BeforeClass**
  在所有测试方法执行前，且仅执行一次。
- **@AfterClass**

执行顺序



JUnit 4 断言机制

常用断言方法

- `assertEquals(expected, actual)`
  验证两个值是否相等。
- `assertTrue(condition)`
  验证条件是否为真。
- `assertFalse(condition)`
  验证条件是否为假。
- `assertNotNull(object)`
  验证对象是否不为 null。

代码示例

```
// 验证加法结果
int result = Calculator.add(1, 2);
assertEquals("1 + 2 应该等于 3", 3, result);
assertTrue("结果应该大于 0", result > 0);
```

若实际结果与预期不符，断言将失败并抛出异常，测试终止。

JUnit 4 异常与超时测试

异常测试 (Exception Test)

验证方法在特定条件下是否会抛出预期的异常。

```
@Test(expected = ArithmeticException.class)
public void testDivideByZero() { ... }
```

超时测试 (Timeout Test)

验证方法的执行时间是否在预期范围内（单位：毫秒）。

```
@Test(timeout = 1000)
public void testLongRunningOperation() { ... }
```

JUnit 4 参数化测试

实现步骤

- 使用`@RunWith(Parameterized.class)`注解测试类。
- 声明变量存放测试参数和预期结果。
- 提供`@Parameters`注解的静态方法，返回测试数据。
- 创建构造函数，接收数据并赋值给变量。
- 编写测试方法，使用变量进行测试。

代码示例

```
@RunWith(Parameterized.class)
public classCalculatorTest {
    private inta, b, expected;
    @Parameters
    public staticCollection<Object[]> data() { … }
    publicCalculatorTest(int a, int b, int expected) { … }
    @Test
    public voidtestAdd() { … }
}
```

JUnit 4 测试套件（Suite）

什么是测试套件？

将多个相关的测试类组织在一起，以便可以一次性运行所有测试，常用于全面回归测试。

如何使用？

- 创建一个空类作为测试套件入口。
- 使用 `@RunWith(Suite.class)` 注解该类。
- 用 `@Suite.SuiteClasses` 指定测试类。

代码示例

```
@RunWith(Suite.class)

@Suite.SuiteClasses({
TestClass1.class, TestClass2.class
})

public class AllTests {}
```

运行 AllTests 类，即可执行所有指定的测试。

JUnit 4运行原理深度解析

探索框架背后的机制

如何运行测试用例

- 测试函数前的 绿色 三角形 运行按钮
- 测试源代码文件，点右键，在菜单中运行
- 测试源代码目录，点右键，在菜单中运行

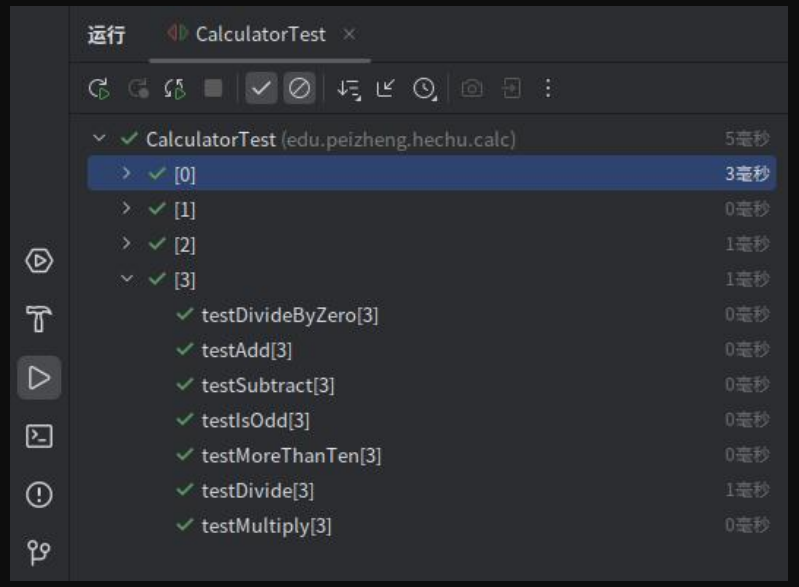
```java
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    @Test
    public void testAdd() {
        assertEquals( expected: 5, Calculator.add( a: 2,  b: 3));
        assertEquals( expected: 0, Calculator.add( a: -2,  b: 2));
        assertEquals( expected: -5, Calculator.add( a: -2,  b: -3));
        assertEquals( expected: 10, Calculator.add( a: 10,  b: 0));
    }

```

- 如果正常会正确运行：



JUnit 4 架构概览



测试接口/测试方法（Test）

所有测试用例的顶层接口，定义了测试执行的基本方法。



测试运行器（TestRunner）

框架的“发动机”，负责加载、执行测试并收集结果。



测试用例（TestCase）

实现Test接口，代表一个具体的、可执行的测试方法。



测试结果（TestResult）

记录测试过程中的事件，如成功、失败、错误等。



测试套件（TestSuite）

将多个Test（TestCase或TestSuite）组织成一组进行批量执行。



断言（Assert）

提供静态方法库，用于验证测试预期结果与实际结果。

核心组件：TestRunner

TestRunner：JUnit的核心执行引擎

解析测试类

识别所有带有 `@Test` 注解的方法。

创建测试实例

为每个测试方法创建一个新的测试类实例。

执行测试方法

按 `@Before`、`@Test`、`@After` 顺序执行。

收集测试结果

捕获异常，记录测试的成功与失败。

工作流程

1. 接收测试类
2. 解析并执行
3. 收集测试结果
4. 生成测试报告

核心组件：TestResult

测试结果的收集者

事件监听器

实现 TestListener 接口，持续监听测试过程中的各类事件。

结果统计

精确记录测试总数、失败数（AssertionError）和错误数（其他 Exception）。

信息查询

提供丰富的 API 方法，用于获取和分析测试统计数据。

核心统计信息



核心组件：TestCase & TestSuite

TestCase

代表一个独立的测试用例，通常对应一个测试方法。

TestSuite

代表一个测试用例的集合，用于组织和批量执行测试，可以嵌套包含。

测试结构示意图



JUnit 扩展机制：Runner

什么是 Runner？

Runner是JUnit 4的核心扩展点，它定义了如何发现和执行测试用例。

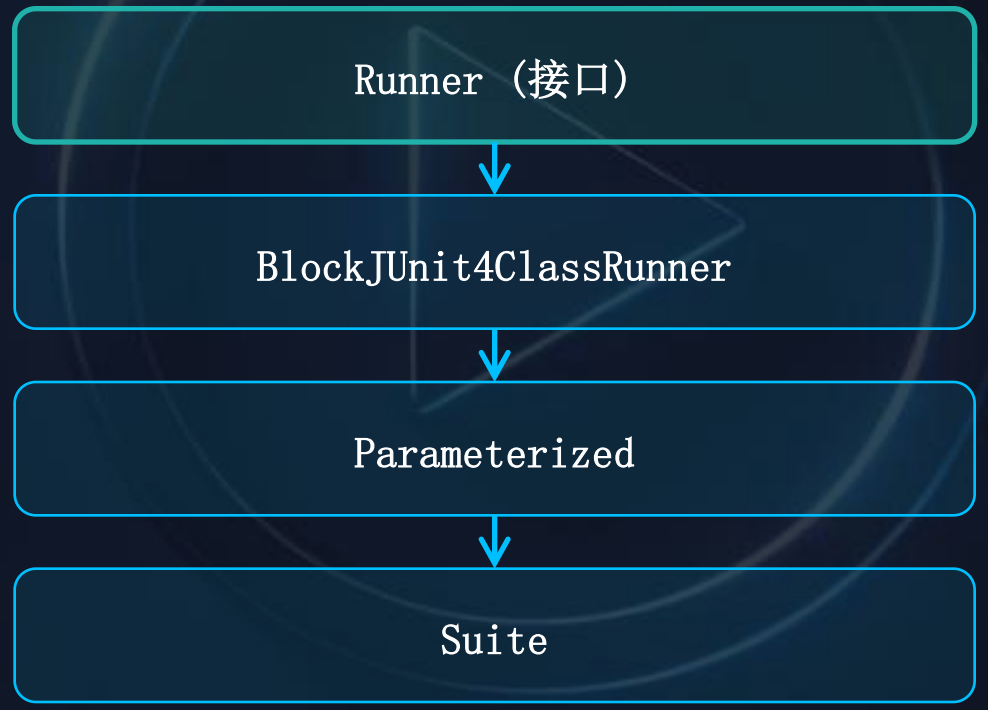
自定义 Runner

通过实现Runner接口或继承现有实现，可以灵活定制测试执行逻辑。

常见 Runner

- BlockJUnit4ClassRunner：JUnit 默认运行器
- Parameterized：用于参数化测试

Runner 接口实现



JUnit 扩展机制：@RunWith

@RunWith 注解作用

指定测试运行器

用于指定测试类的运行器（Runner），告诉JUnit如何执行测试。

简单使用方式

在测试类上添加 @RunWith(MyCustomRunner.class) 即可。

实现扩展的核心

替换Runner以改变测试执行行为，是JUnit 扩展的主要方式。

代码示例：参数化测试

```
@RunWith(Parameterized.class)
public classMyTest {
    private finalint input;
    private finalint expected;
}
```

通过指定 Parameterized Runner，实现多组输入的自动化测试。

JUnit 执行流程（1）



以上是JUnit 4执行流程的前半部分，涵盖了从启动到准备执行测试方法的关键步骤。

JUnit 执行流程（2）

循环执行每个测试方法



创建新的测试类实例



执行@Before方法



执行@Test方法并记录结果



执行@After方法

最终清理与报告



执行@AfterClass方法（一次）



根据 TestResult 生成测试报告



流程结束

JUnit 监听器（Listeners）

什么是监听器？

JUnit 4提供了监听器机制，允许开发者在测试执行的生命周期中插入自定义的逻辑，如日志记录、性能监控或失败处理等。

核心接口：TestListener

该接口定义了一系列回调方法，对应测试执行的不同阶段。通过实现此接口，即可对测试事件做出响应。

常用回调方法

testRunStarted()

测试套件开始执行时调用。

testRunFinished()

测试套件执行完成时调用。

testStarted()

单个测试方法开始执行时调用。

testFinished()

单个测试方法执行完成时调用。

testFailed()

单个测试方法执行失败时调用。

代码覆盖率

度量测试的有效性

代码覆盖率定义与价值

定义

代码覆盖率（Code Coverage）是一种度量指标，用于衡量在测试过程中，被执行的代码占总代码的比例。

核心价值

评估测试完整性

帮助开发者了解测试用例对代码的覆盖程度。

发现未测试代码

找出潜在风险点，避免测试盲区。

指导测试优化

为测试用例的编写和优化提供数据支持。

代码示例

```
function calculate(a, b) {
    if (a > b) return a + b;
    else return a * b;
}
```

- 已覆盖代码（Tested）
- 未覆盖代码（Untested）

覆盖率指标：行覆盖率

行覆盖率（Line Coverage）

最常用的覆盖率指标，衡量被执行的代码行数占总代码行数的比例。

计算公式

$$(\text{被执行行数} / \text{总行数}) \times 100\%$$

特点

计算简单，易于理解，但无法衡量代码的分支和逻辑覆盖情况。

代码示例

```
function add(a, b) {
return a + b; // 被执行
console.log("未被执行"); // 未执行
}
```

计算结果

行覆盖率 = 1 / 2 =50%

覆盖率指标：分支覆盖率

什么是分支覆盖率？

也叫判定覆盖率，衡量程序中每个条件判断的取真和取假分支是否都被执行到。

计算公式

（被执行的分支数 / 总分支数）× 100%

核心特点

- 比行覆盖率更精确，能发现逻辑漏洞。

代码示例

```
function isEven(num) {
    if (num % 2 === 0) {
        return true; // 分支1: 已覆盖
    } else { return false; } // 分支2: 未覆盖
```

分支覆盖率 = （1 / 2）× 100% =50%

覆盖率指标：方法覆盖率

方法覆盖率（Method Coverage）

衡量被测试调用过的方法数量占总方法数量的比例。

计算公式

（被调用的方法数 / 总方法数）× 100%

特点

从宏观层面衡量测试覆盖，关注整个方法是否被执行。

示例：类方法覆盖情况

Class: UserService

- ✓ login(username, password)
- ✓ getUserInfo(userId)
- ✗ deleteUser(userId)

方法覆盖率:66.7%

覆盖率指标：类覆盖率

类覆盖率（Class Coverage）

衡量被测试加载并执行过的类数量占总类数量的比例。

计算公式

类覆盖率 =（被加载的类数 / 总类数）× 100%

特点

- 从最宏观的层面衡量测试覆盖情况。
- 关注整个类是否被使用，快速定位未触及的代码。

覆盖情况示例

com.example.project（包）

- ✓ UserService.java（已覆盖）
- ✓ OrderRepository.java（已覆盖）
- ✗ PaymentGateway.java（未覆盖）
- ✗ ReportGenerator.java（未覆盖）

示例覆盖率：2 / 4 =50%

主流覆盖率工具介绍



JaCoCo

开源免费，功能强大，易于集成。支持多种覆盖率指标，是目前最流行的Java代码覆盖率工具。



Cobertura

开源免费，支持多种覆盖率指标，并能生成详细的HTML报告，稳定可靠。



Emma

较早的覆盖率工具，开源免费，功能相对简单，但因其轻量，仍有部分项目在使用。

在接下来的内容中，我们将重点介绍 JaCoCo 的使用。

JaCoCo 集成与使用（1） - Maven

集成步骤

添加JaCoCo插件依赖

在 `pom.xml` 的 `plugins` 部分添加 `jacoco-maven-plugin`。

配置插件目标

- prepare-agent:测试前启动JaCoCo代理，收集数据。
- report:测试后生成HTML格式的覆盖率报告。

配置示例（pom.xml）

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.10</version>
    <executions>...</executions>
</plugin>
```

运行命令:mvn clean test

JaCoCo 集成与使用（2） - 查看报告

生成报告

执行 `mvn clean test` 命令，JaCoCo 会在 `target/site/jacoco/` 目录下生成 HTML 格式的覆盖率报告。

解读报告

项目总览

展示项目整体的行、分支等覆盖率指标。

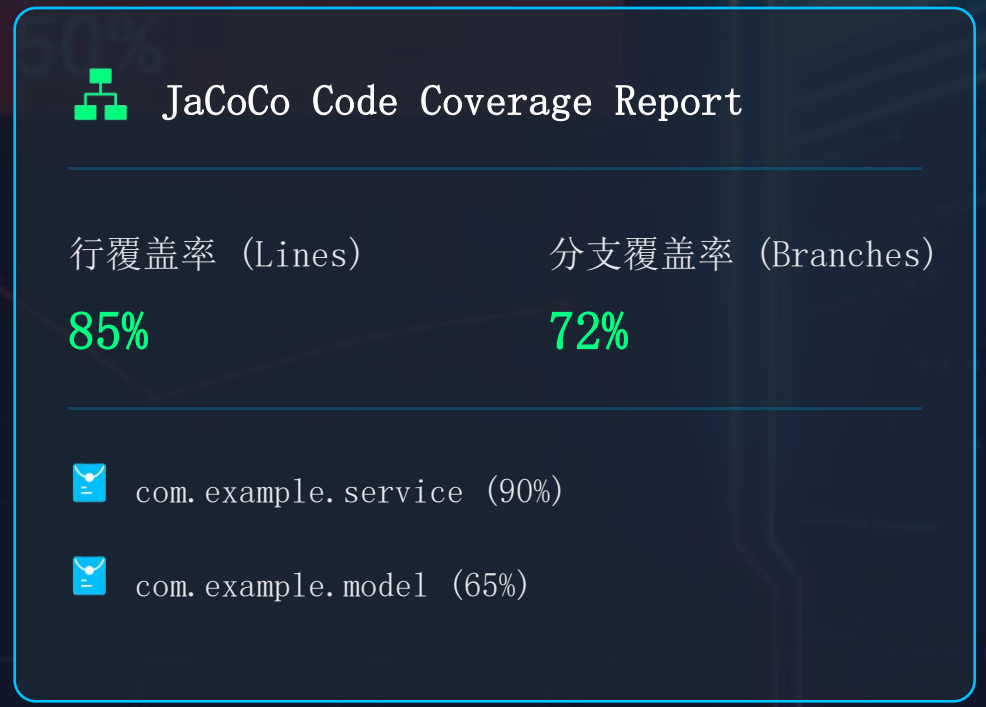
层级视图

从包、类到方法，层层深入查看细节。

源码高亮

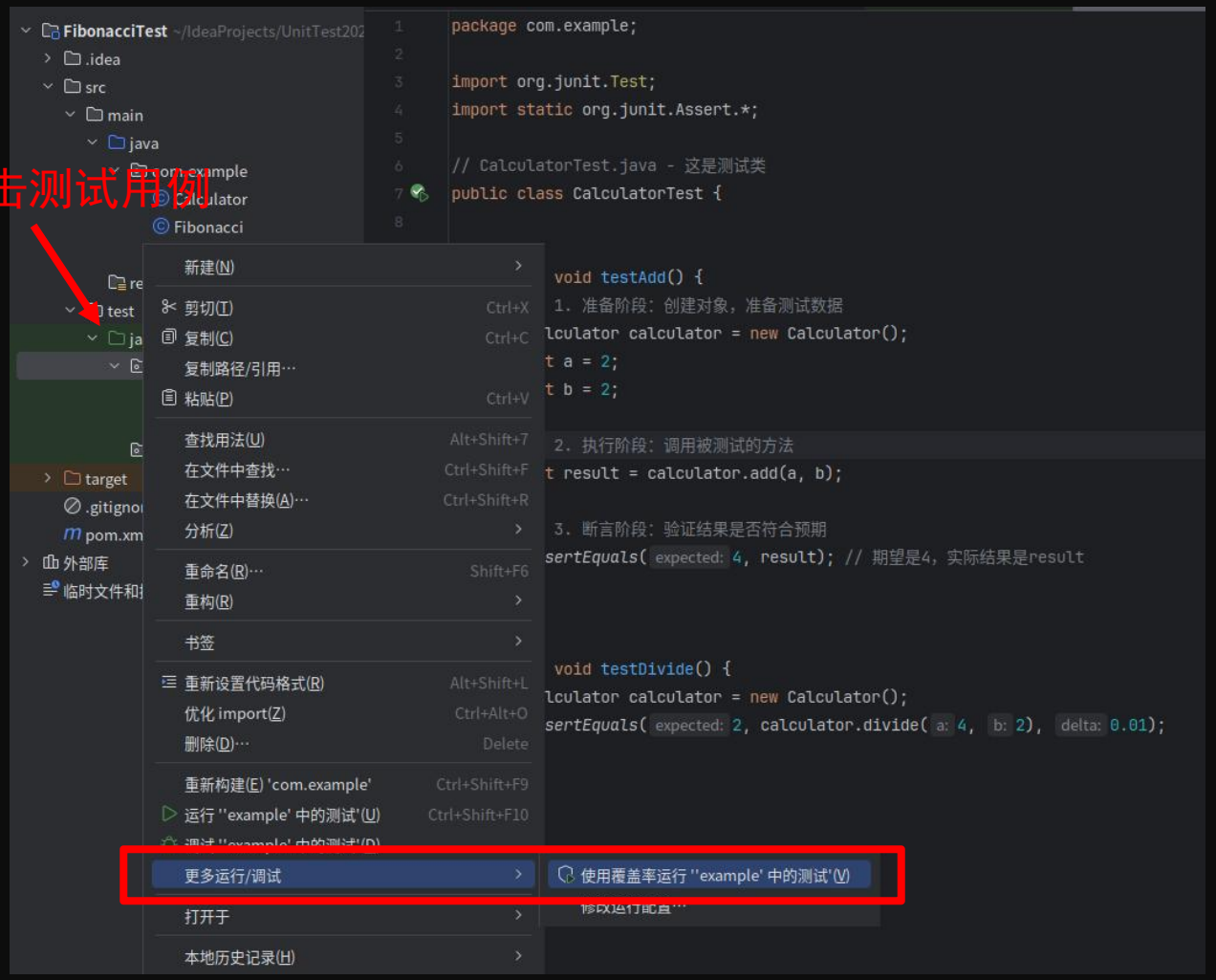
在源码中直观标记覆盖（绿）与未覆盖（红）行。

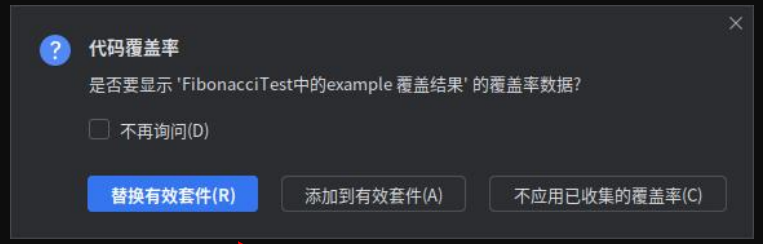
报告预览



覆盖率测试

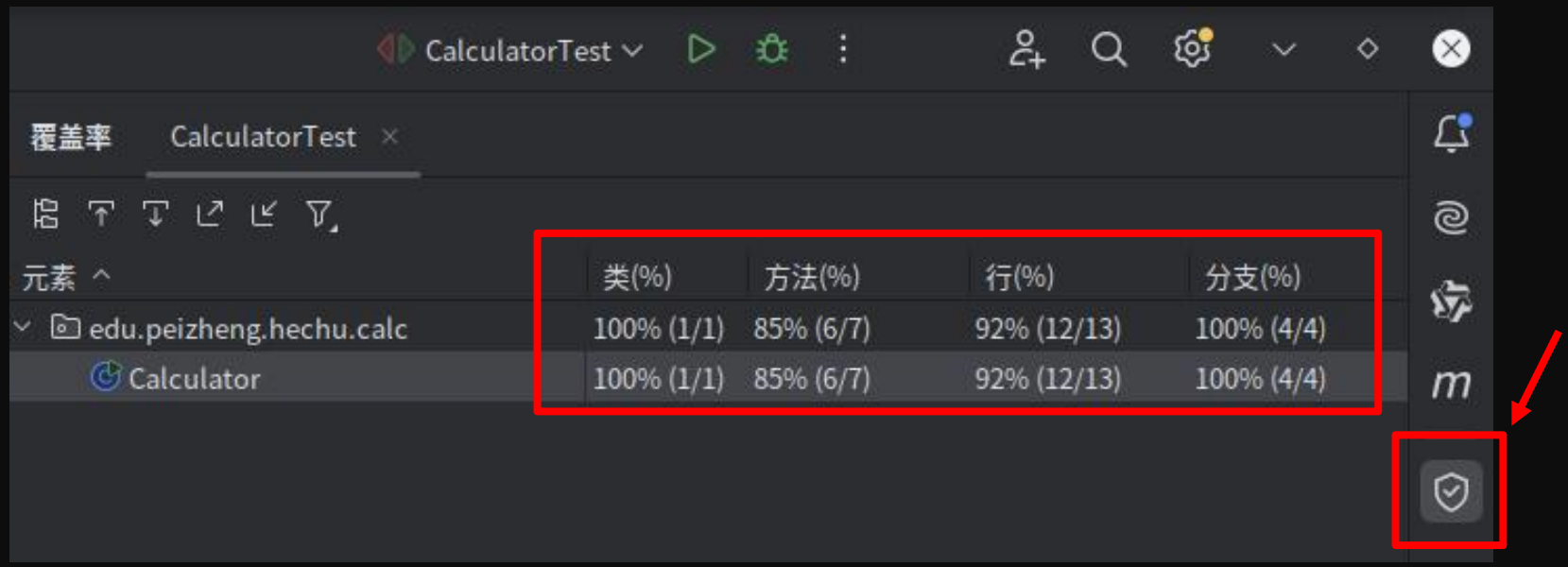
右键点击测试用例





如果出现提示框，选择“替换有效套件”

查看覆盖率结果


CalculatorTest
覆盖率
CalculatorTest
元素
类(%)
方法(%)
行(%)
分支(%)
edu.peizheng.hechu.calc
100% (1/1)
85% (6/7)
92% (12/13)
100% (4/4)
Calculator
100% (1/1)
85% (6/7)
92% (12/13)
100% (4/4)

分支覆盖率

绿色：测试用例已经覆盖

黄色：条件判断运行

红色：测试用例未覆盖

```java
// Calculator.java - 这是我们要测试的类
public class Calculator {  4个用法

    // 我们要测试的方法：加法
    public int add(int a, int b) {  1个用法
        return a * b ;
    }

    // 另一个要测试的方法：除法
    public double divide(int a, int b) {  1个用法
        if (b == 0) {
            throw new IllegalArgumentException("除数不能为零");
        }
        return (double) a / b;
    }
}
```

代码覆盖率的误区与最佳实践

常见误区

追求100%覆盖率

100%覆盖率不代表无bug，过度追求会导致测试成本激增。

覆盖率越高越好

覆盖率只是参考，高覆盖率背后可能是大量无效测试。

忽视测试质量

相比于测试数量，发现问题的能力更为重要。

最佳实践

关注业务核心逻辑

优先保证核心业务代码的高覆盖率，投入产出比最高。

结合其他质量指标

综合评估代码复杂度、缺陷密度等，全面度量质量。

持续改进而非考核

将覆盖率作为发现盲点、驱动改进的工具，而非目标。

变异测试

更严苛的测试质量评估

什么是变异测试？

定义

变异测试（Mutation Testing）是一种评估测试用例质量的技术。它通过对源代码进行微小的、语义保持的修改（即“变异”），生成多个“变异体”（Mutant）。
对源代码进行修改（变异）由程序完成，模拟人编写代码时犯错。

核心思想

运行测试用例检查变异体。若测试用例能发现变异（执行结果不同），则变异体被“杀死”（Killed）。好的测试用例应能杀死尽可能多的变异体。

代码变异示例

原始代码

```
int result = a + b;
```

变异体（Mutant）

```
int result = a × b;
```

结果

若测试用例能发现此差异，则变异体被“杀死”

变异测试的核心思想



变异分数 =（被杀死的变异体数量 / 总变异体数量）× 100%，是衡量测试用例集质量的关键指标。

变异算子（Mutation Operators）

算术运算符替换

将 +，-，*，/ 等算术运算符进行互换。

a + b → a - b

关系运算符替换

将 >，<，==，!= 等关系运算符进行互换。

a > b → a <= b

逻辑运算符替换

将 &&（与）和 ||（或）逻辑运算符进行互换。

a && b → a || b

条件取反

将 if (condition) 替换为 if (!condition)。

变量替换

将一个变量替换为另一个同类型的变量。

pom.xml 文件添加Pitest

```
<build>
    <plugins>
        <plugin>
            <groupId>org.pitest</groupId>
            <artifactId>pitest-maven</artifactId>
            <version>1.1.10</version>
            <configuration>
                <targetClasses>
                    <param>edu.peizheng.hechu.calc.*</param>
                </targetClasses>
                <targetTests>
                    <param>edu.peizheng.hechu.calc.*</param>
                </targetTests>
                <mutators>
                    <mutator>INCREMENTS</mutator>
                    <mutator>RETURN_VALS</mutator>
                    <mutator>INVERT_NEGS</mutator>
                    <mutator>CONDITIONALS_BOUNDARY</mutator>
                    <mutator>MATH</mutator>
                    <mutator>VOID_METHOD_CALLS</mutator>
                    <mutator>NEGATE_CONDITIONALS</mutator>
                </mutators>
            </configuration>
        </plugin>
    </plugins>
</build>
```

输出变异杀死html报告

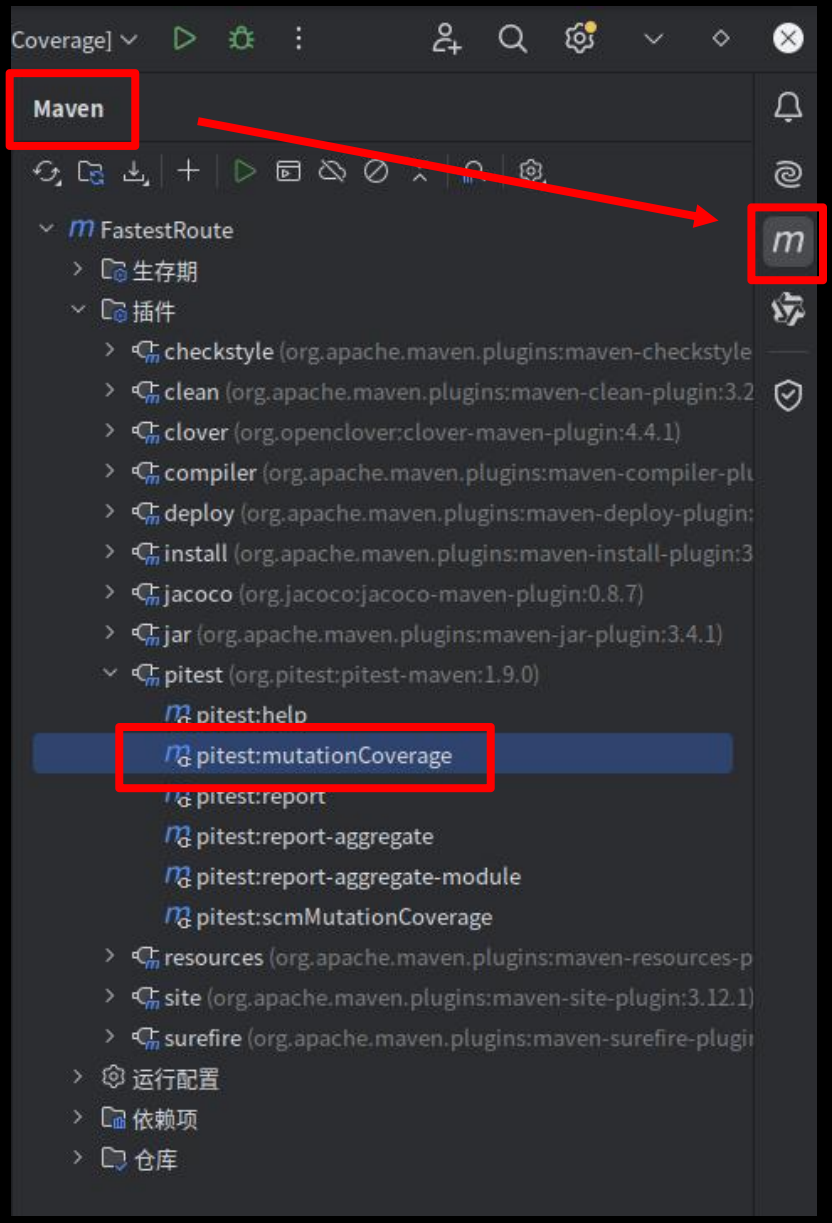
注意包名要使用自己的包名

变异率测试

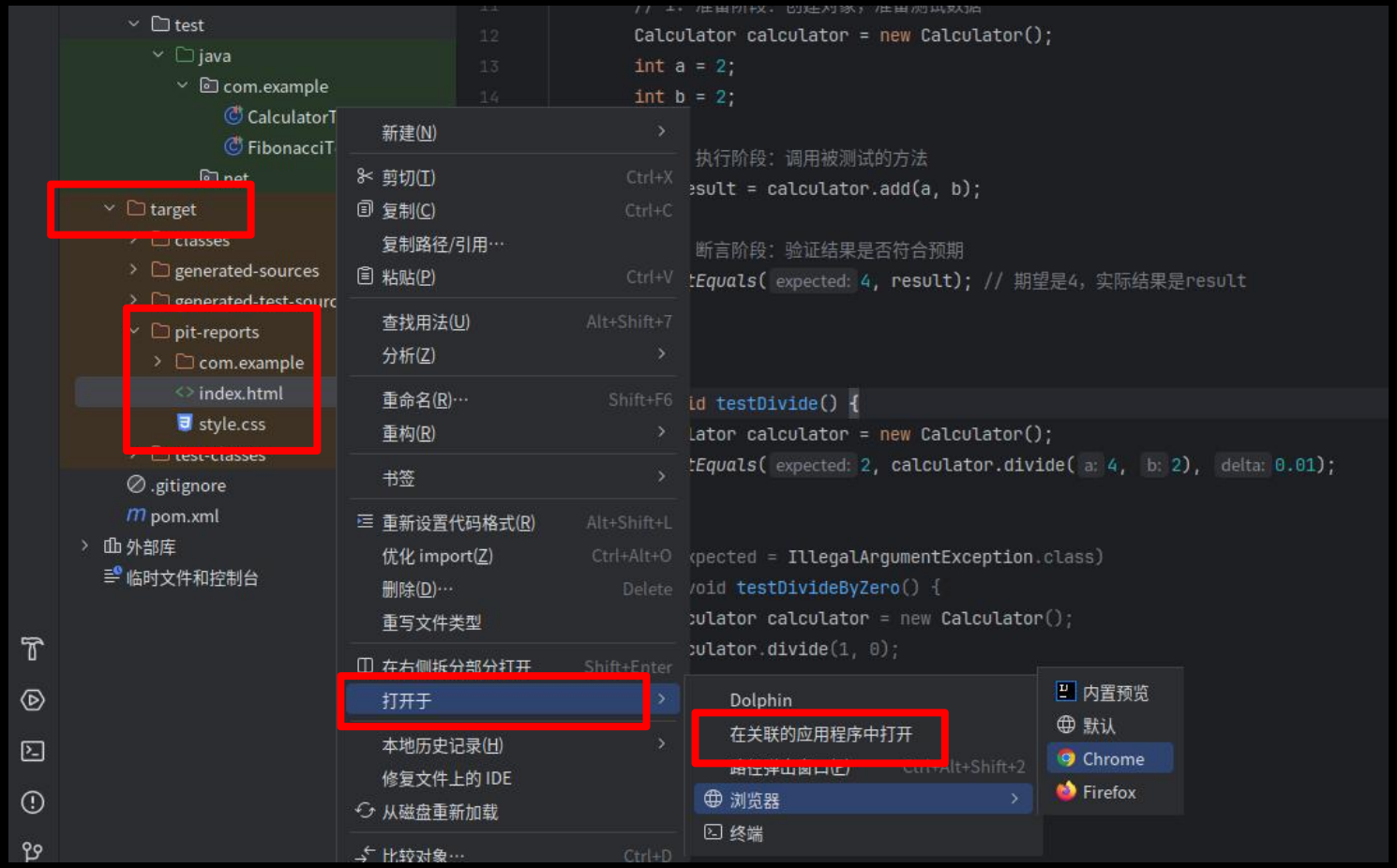
打开 Maven 面板

选择“插件”

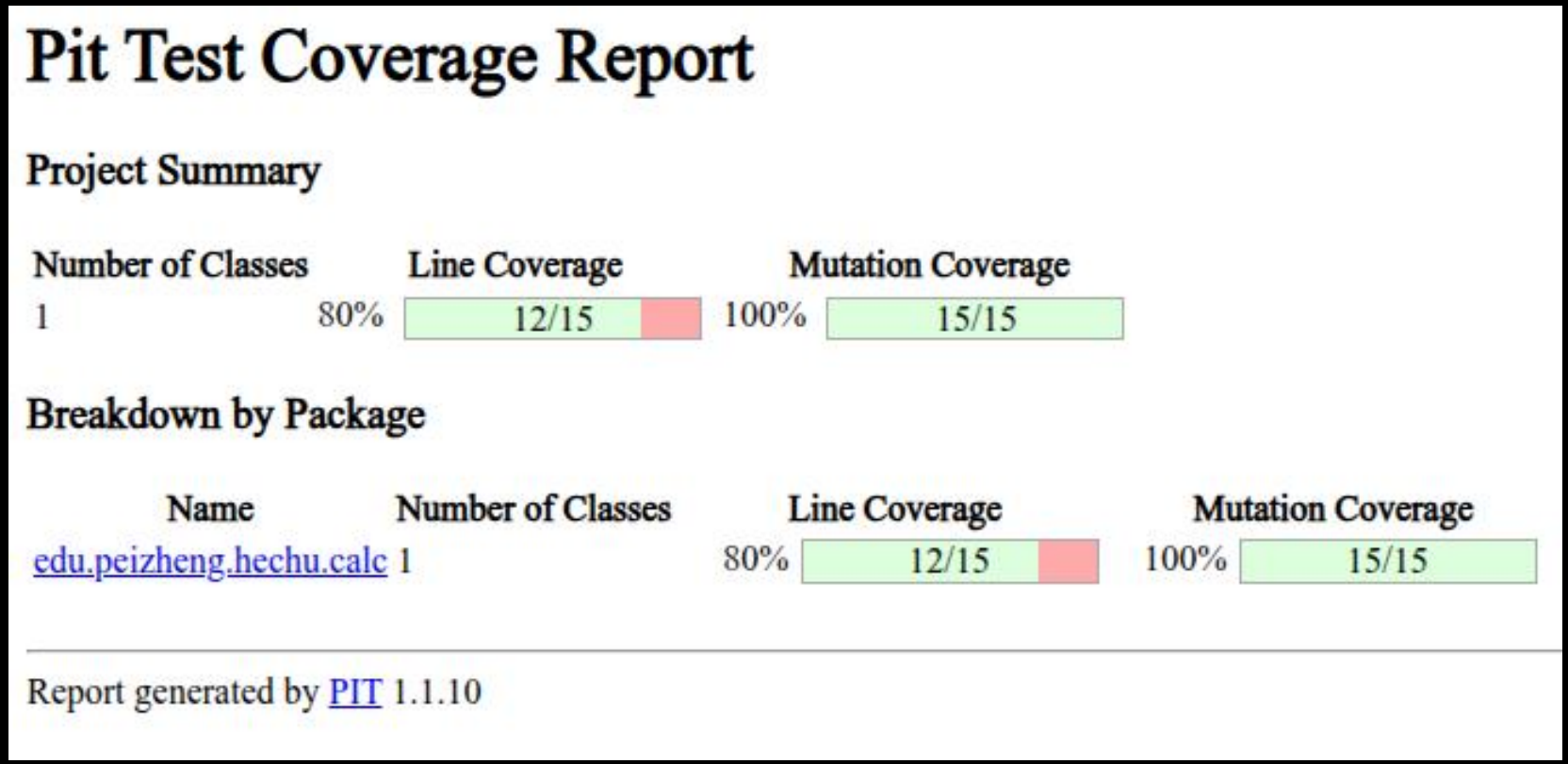
pitest:mutationCoverage



查看变异杀死率


test
java
com.example
target
generated-sources
pit-reports
com.example
index.html
style.css
.gitignore
pom.xml
外部库
临时文件和控制台
新建(N)
剪切(T) Ctrl+X
复制(C) Ctrl+C
复制路径/引用…
粘贴(P) Ctrl+V
查找用法(U) Alt+Shift+7
分析(Z)
重命名(R)… Shift+F6
重构(R)
书签
重新设置代码格式(R) Alt+Shift+L
优化 import(Z) Ctrl+Alt+O
删除(D)… Delete
重写文件类型
打开于
本地历史记录(H)
修复文件上的 IDE
从磁盘重新加载
Dolphin
在关联的应用程序中打开
浏览器
终端
内置预览
默认
Chrome
Firefox
Calculator calculator = new Calculator();
int a = 2;
int b = 2;
执行阶段：调用被测试的方法
断言阶段：验证结果是否符合预期
testDivide()
testDivideByZero()

查看变异杀死率



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage	
1	80%	12/15	100%	15/15

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage	
edu.peizheng.hechu.calc	1	80%	12/15	100%	15/15

Report generated by PIT 1.1.10

变异分数（Mutation Score）

定义

衡量测试用例集杀死变异体能力的核心指标。

计算公式

变异分数 =（被杀死的变异体 /（总变异体 − 等价变异体））× 100%

等价变异体

代码修改后语义不变的变异体，无法被杀死，计算时需排除。



意义

分数越高，测试用例质量越高，发现Bug的能力越强。

变异测试的优缺点

优点

评估测试用例质量

比代码覆盖率更能反映测试发现bug的能力。

发现测试用例弱点

存活的变异体指示了测试未覆盖的逻辑场景。

指导测试用例优化

通过分析存活变异体，有针对性地补充测试。

缺点

计算成本高

生成和运行大量变异体需耗费大量时间资源。

等价变异体问题

需人工分析识别，增加额外工作量。

对测试用例要求高

需要测试用例能精确验证程序行为。

案例与代码示例

理论联系实际

案例：单元测试与覆盖率分析

案例需求

实现计算器类
包含加、减、乘、除、平方等基本运算。

编写单元测试
为Calculator类编写全面的测试用例。

生成覆盖率报告
使用JaCoCo工具生成并分析报告。

实践流程

- 编写Calculator业务代码
- 使用JUnit编写测试用例
- 在Maven项目中集成JaCoCo
- 运行测试并生成覆盖率报告
- 分析报告并优化测试用例

代码示例：被测类 Calculator

```
public class Calculator {
    public int add(int a,int b) {return a + b; }
    public int subtract(int a,intb) {return a - b; }
    public int multiply(int a,intb) {return a * b; }
    public double divide(int a,int b) {
        if(b == 0) {throw new IllegalArgumentException("除数不能为零"); }
        return (double) a / b; }
    public int square(int a) {return a * a; }
}
```

注意：除法方法 `divide` 中包含了除数为零的异常处理，这是后续异常测试的关键场景。

代码示例：JUnit 测试用例

```
importorg.junit.Before;
importorg.junit.Test;
import staticorg.junit.Assert.*;
public classCalculatorTest {
  privateCalculator calculator;
  @Before
  public void setUp() { calculator =new Calculator(); }
  @Test
  public void testAdd() { assertEquals(5, calculator.add(2, 3)); }
  @Test
  public void testSubtract() { assertEquals(-1, calculator.subtract(2, 3)); }
  @Test(expected = IllegalArgumentException.class)
  public void testDivideByZero() { calculator.divide(6, 0); }
}
```

通过JUnit 4注解与断言，全面验证Calculator类的正确性，确保代码质量。

代码示例：JaCoCo 覆盖率报告解读

配置 JaCoCo

在 `pom.xml` 中添加并配置 `jacoco-maven-plugin` 插件。

运行测试并生成报告

执行 `mvn clean test` 命令，自动生成 HTML 报告。

解读报告

- 行覆盖率与分支覆盖率均达100%
- 源码视图中所有行均被绿色标记
- 结论：测试覆盖了所有代码行与逻辑分支

JaCoCo HTML 报告示例

项目总览: Calculator 类

行覆盖率	100%
分支覆盖率	100%

所有代码行和逻辑分支均被测试覆盖，包括异常处理场景。

pom.xml 文件添加Jacoco

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.7</version>
    <executions>
        <execution>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

可以生成代码覆盖率的html报告

案例总结与思考

实践收获

- 掌握JUnit 的基本用法，包括注解、断言和异常测试。
- 学会为简单类编写全面的单元测试用例。
- 在Maven项目中集成JaCoCo并生成覆盖率报告。
- 能够解读覆盖率报告，并评估测试的完整性。

扩展思考

- 如何为更复杂的业务逻辑（如三角函数）设计测试用例？
- 除了覆盖率，还有哪些指标（如变异测试）能评估测试质量？
- 如何将单元测试和覆盖率分析集成到CI/CD流程中？

总结与展望

回顾与前行

总结：单元测试核心价值



质量保障

单元测试是软件质量的第一道防线，能在早期发现并修复缺陷，减少后期维护成本。



开发效率

作为活文档帮助理解代码，为重构提供安全网，极大提升开发与迭代效率。



团队协作

确保代码提交质量，减少因质量问题导致的团队内耗，提升整体协作效率。

单元测试：构建高质量软件的基石

总结：JUnit 关键特性回顾

注解驱动

使用`@Test`, `@Before`, `@After`等注解，简化了测试代码的编写。

丰富的断言

提供了`Assert`类，包含一系列静态方法，用于验证测试结果。

异常测试

支持通过`@Test(expected = Exception.class)`测试方法是否抛出预期异常。

超时测试

支持通过`@Test(timeout = 1000)`来测试方法的执行时间。

参数化测试

使用`@Parameters`实现用不同参数多次运行同一个测试方法。

总结：质量度量的意义

数据驱动决策

提供客观、量化的数据支持，帮助我们做出更科学的决策。

持续改进

发现过程薄弱环节，进行有针对性的改进，提升软件质量。

风险识别

提前识别潜在质量风险，如低覆盖率、高复杂度代码，防患于未然。

沟通协作

统一的度量标准为团队提供共同语言，便于达成质量共识。

